Arbitrage-Free Neural SDE Model for Pricing a Book of European Options

Laurent Martini–Bonnet

February 2026

Contents

1 Introduction **2**

2 Approach and motivation **3**

3 Market model **4**
3.1 Structure 4
3.2 Options lattice 4
3.3 Modelling of latent factor dynamics 5

4 Arbitrage constraints **6**
4.1 Static arbitrage 6
4.2 Dynamic arbitrage 12

5 Decoding the factors **15**
5.1 Setting and objectives 15
5.2 Algorithm 17

6 Neural SDE model **20**
6.1 A constrained supervised learning problem 20
6.2 Modelling the constraints 23
6.3 Example of a diffusion constrained in a polytope 26

7 Implementation of the model **28**
7.1 Choice of the Heston–SLV Parameters and calibration 28
7.2 Synthetic Data Generation Procedure 29
7.3 Generation of option prices 30
7.4 Training Phase (Learning the Dynamics) 31

8 Validation and tests **33**
8.1 In-Sample Test 33
8.2 Out-of-sample test 35
8.3 Critique of the approach 39

This paper revisits the original article "Arbitrage-Free Neural-SDE Market Models" by Cohen, Reisinger, and Wang with a primarily pedagogical perspective, while providing some additional insights and personal contributions.

1 Introduction

The goal of this paper is to develop a model for a European options book, i.e., to price all of the options contained in this book. The pricing of options is important because without a pricing framework, buyers and sellers would not know what a fair price is, and that would allow arbitrage opportunities (see [1] page 174 for an example). Furthermore, the pricing of all options contained in a book is much more informative than one of a single option, as the prices of different options are linked: for example, it is known that under any model, the price of a call is decreasing with respect to the strike price. Hence, beyond the pricing of individual instruments, the fundamental challenge lies in capturing the joint dynamics of the underlying asset S and the entire surface of option prices. We aim for a model which is arbitrage free, that replicates stylised facts observed from real market prices, and that computes exactly the prices of all options in the Option book. We also want the model to be practically estimated.

2 Approach and motivation

The approach considered here consists of decoding factors from option prices directly: that is to say, the option prices are expressed as linear combinations of d latent factors. The joint process $(S_t, \xi_t)_{0 \leq t \leq T}$, with $S_t \in \mathbb{R}$ the underlying asset price at time t and $\xi_t \in \mathbb{R}$ the latent factors at time t, will be then represented as a diffusion process.
The motivation behind using a diffusion process is to be able to apply stochastic calculus and the fact that numerous financial models to price options are diffusion models (Black-Scholes, Dupire, Heston, rough path volatility models). Henceforth, we define $X_t = (S_t, \xi_t)$ and we assume that there exists μ, σ and a d+1-dimensional brownian motion W such that:

$$dX_t = \mu(X_t, t)dt + \sigma(X_t, t)dW_t$$

The goal will be to approximate μ, σ with neural networks. The motivation to use these is that as neural networks can approximate any integrable function ([2]), we let the data choose the best dynamic, hence we have a lot of flexibility. Moreover, calibration and model selection are done in one step. Indeed, the data selects the parameters and the nature of the model will then emerge: the selection of the parameters and of the structure of the model are done at the same time. Whereas in the Heston model, for instance, the structure of the model is chosen first and then, we optimize the parameters. Furthermore,as demonstrated in [3], neural networks can produce robust estimates of derivatives prices. In our setting, this helps to deal with the inherent incompleteness of the market. Indeed, the Equivalent Martingale Measure (EMM) $\mathbb{Q}$ is not unique; there exists an infinite family of such measures. The Neural SDE approach provides a data-driven resolution to this problematic. Instead of imposing a specific parametric measure, the neural network "learns" from time-series data the specific measure $\mathbb{Q}$ that the market effectively selects to price and hedge options. However, choosing neural networks also induces some challenges: they are naturally less interpretable as there are too many parameters for them to have an individual meaning. They are also very demanding in data, and as shown in [4], optimization on SDE is computationnally difficult.

3 Market model

3.1 Structure

We assume a frictionless market, i.e without transaction costs and liquidity constraints. The dividend at time t, q_t, is the amount of cash one will receive if one owns a share of the stock at time t. We also define d(t,T) as the value at time t of one unit of cash which will be received at time T. We then define interests rates by the relation: $d(t,T) = e^{-\int_t^T r_s ds}$. The dividend factor is defined by: $\Gamma_t(T) = e^{\int_t^T q_s ds}$. We can then define the forward price as: $F_t(T) = \frac{S_t}{\Gamma_t(T) d_t(T)}$, which is the price of which one agrees at time t to buy S at time T. We allow interest rates and dividends to be non-constant but deterministic.

We focus on the joint dynamics of liquid options only, i.e options which are not far from the money (the strike price is not too far from the current price of the stock, and expiry is not too close).
Note: Options whose maturity is too close are not liquid in general because the strike is not too different from the current price of the stock, hence it does not make much sense to buy the option. If the strike is much different from the stock price, as in general we don't expect much movement in a short time, then very few people will want to purchase the option.

3.2 Options lattice

We consider a collection of available European call prices. We represent the range of liquid options in (T,K)-coordinates, with T the maturity, and K the strike price. However, as we just saw, liquidity varies over time. Hence we use (τ_t, m_t) as coordinates, where $\tau_t = T - t$, $m_t = log(\frac{K}{F_t(T)})$. τ_t represents the time to maturity and m_t measures the spread between the strike price and the forward price of the current price of the stock. m_t is called the moneyness at time t. The log is added for scale reasons.

We assume that the range of liquid options remains constant, i.e, the shape of the grid does not change over time. However, an option can be in the liquid range at one time before maturity and not at another time. We define the liquid range as the following: τ_{inf},τ_{sup} and m_{inf}, m_{sup} as the minimum and maximum time to maturity and moneyness among options within the liquid range. We then define: $R_{liq} = \{(\tau, m) \in \mathbb{R}^2 : \tau \in [\tau_{inf}, \tau_{sup}], m \in [m_{inf}, m_{sup}]\}$
In practice, we construct R_{liq} by interpolation of a finite collections of known call option prices, which we will call L_{liq} and define as:

$$\mathcal{L}_{\text{liq}} = \{(\tau_i, m_j^i) \in \mathcal{R}_{\text{liq}} : \underline{m}(\tau_i) = m_1^i < \cdots < m_{n_i}^i = \overline{m}(\tau_i), \underline{\tau} = \tau_1 < \cdots < \tau_p = \overline{\tau}, 1 \leq j \leq n_i, 1 \leq i \leq p\}.$$

We try to choose interpolation points as far from each other as possible, to ensure to be able to reproduce the dynamics across the whole R_{liq}. However,

close time to maturity and close moneyness correlate prices with each other. Hence, we can decompose the prices into a few relevant factors, built from the data. For instance, we know that option prices depend on S but also on other factors such as implied volatility and volatility skew.

3.3 Modelling of latent factor dynamics

The vector ξ_t serves as the latent "engine" driving the option book. By projecting the high-dimensional price data onto a lower-dimensional state space (e.g., $d = 2$), we posit that the apparent complexity of the market is effectively governed by a few fundamental sources of risk. If ξ is properly identified, these two or three risk sources are sufficient to reconstruct the dynamics of the entire price lattice while ensuring cross-sectional consistency.
We then model the joint dynamics of the price of the underlying asset and the latent factors as:

$$\begin{cases} dS_t = \left(\alpha(\tilde{\xi}_t) - q_t\right) S_t \, dt + \gamma(\tilde{\xi}_t) S_t \, dW_{0,t}, & S_0 = s_0 \in \mathbb{R}; \\ d\xi_t = \mu(\tilde{\xi}_t) \, dt + \sigma(\tilde{\xi}_t) \, dW_t, & \xi_0 = \zeta_0 \in \mathbb{R}^d, \end{cases} \tag{1}$$

where $\tilde{\xi}_t = (S_t, \xi_t)$. We use α_t, γ_t, μ_t and σ_t to denote $\alpha(\tilde{\xi}_t)$, $\gamma(\tilde{\xi}_t)$, $\mu(\tilde{\xi}_t)$ and $\sigma(\tilde{\xi}_t)$, respectively. We denote by $L^p_{\text{loc}}(\mathbb{R}^d)$ the space of all $\mathbb{R}^d$-valued, progressively measurable, and locally p-integrable (in t, $\mathbb{P}$-a.s.) processes on $[0, T^*]$, and assume that $\alpha S \in L^1_{\text{loc}}(\mathbb{R})$, $\mu \in L^1_{\text{loc}}(\mathbb{R}^d)$, $\gamma S \in L^2_{\text{loc}}(\mathbb{R})$ and $\sigma \in L^2_{\text{loc}}(\mathbb{R}^d)$.

We can now define the call price surface: $C_t : (T, K) \to C_t(T, K)$
We also define two normalizations: $\hat{c}_t(T, K) = \frac{C_t(T,K)}{D_t(T)F_t(T)}$, $\tilde{c}(T-t, M(K; F_t(T))) := \hat{c}_t(T, K)$.
We want to model:
$(\tau, m) \to \tilde{c}_t(\tau, m)$ where we abuse the notations for τ_t, m_t.
We would now like to link the option prices with the latent factors. We will assume the relation:

$$\tilde{c}_t(\tau, m) = g(t, \tau, m, \xi_t) := G_0(\tau, m) + \sum_{i=1}^{d} G_i(\tau, m)\xi_{it}, \tag{2}$$

We justify this choice as follows: a linear relation leads to easy estimations and reconstructions of the prices. Moreover, thanks to Weierstrass theorem, we can approximate any smooth function by polynomials and we can make polynomials appear by increasing the number of factors (e.g, if we need ${\xi_1}^3$, we just define $\xi_2 = {\xi_1}^2$, $\xi_3 = {\xi_1}^3$). Lastly, as we will see, the arbitrage constraints are linear in the option prices, so that they will stay linear in the factors thanks to the linear representation, leading to easy verifications that the factors satisfy the constraints.

4 Arbitrage constraints

We will distinguish between two types of arbitrage, the static arbitrage and the dynamic arbitrage.

4.1 Static arbitrage

As defined in [5], a static arbitrage is any arbitrage exploitable by fixed positions on options and the underlying stock at initial time. The position of the underlying stock can be modified at a finite number of trading times in the future.

Example:
Let us consider two calls with maturity T, one with strike K_1 and the other with strike $K_2, K_1 < K_2$. Denote by C_1 and C_2 their respective prices and assume $C_2 > C1$. Then if you buy the call at price C_1 and that you sell the call at price C_2, you make an initial profit of $C_2 - C_1 > 0$. Moreover, your payoff at maturity is $(S_T - K_1)^+ - (S_T - K_2)^+$, which is non-negative as $K_1 < K_2$.

Note: As static arbitrage does not depend on the model used, it is also known as "model-free arbitrage".

Absence of static arbitrage is a simpler condition to achieve than absence of arbitrage, as the set of static arbitrage opportunities is typically smaller than the set of arbitrage opportunities. Now to enonciate some sufficient conditions to achieve absence of static arbitrage, we need a few definitions.

Definition:
Let $q(x,t)$ be a family of probability densities on $\mathbb{R}$, and denote by μ_t the associated probability measures.
We say that $\{q(\cdot,t)\}_{t\geq 0}$ satisfies the *martingale marginal property* if there exists

- a filtered probability space $(\Omega, \mathcal{F}, (\mathcal{F}_t)_{t\geq 0}, \mathbb{P})$,
- an $(\mathcal{F}_t)$-martingale $(M_t)_{t\geq 0}$,

such that for all $t \geq 0$,

$$M_t \sim q(\cdot, t),$$

i.e. M_t admits density $q(\cdot,t)$.

Now we have this very deep result, proven in [6]:

Theorem: The absence of static arbitrage is equivalent to the fact that the family of risk neutral densities has the martingale marginal property.

This martingale marginal property seems rather absract, and hard to check in practice. The so-called Kellerer theorem will be useful for this matter, as it links martingale marginal property with convex order, a more natural concept.

Definition: Let μ and ν be two probability measures on $\mathbb{R}$ with finite first moments. We say that μ is smaller than ν in the convex order, denoted $\mu \preceq_{cx} \nu$, if for any convex function $\phi : \mathbb{R} \to \mathbb{R}$ such that the expectations exist, we have:

$$\int_{\mathbb{R}} \phi(x)\, \mu(dx) \leq \int_{\mathbb{R}} \phi(x)\, \nu(dx).$$

In terms of random variables, if $X \sim \mu$ and $Y \sim \nu$, this is equivalent to:

$$\mathbb{E}[\phi(X)] \leq \mathbb{E}[\phi(Y)].$$

Theorem (Kellerer): Let $(\mu_t, t \geq 0)$ be a family of probability measures. Then: μ has the martingale marginal property if and only if μ_t are increasing in the convex order with respect to t.

We construct an example of measures increasing in convex order and highlight an example of the martingale marginal property in this case.

Let

$$X_1 \sim N(0, \sigma_1^2), \qquad X_2 \sim N(0, \sigma_2^2),$$

with $\sigma_1^2 \leq \sigma_2^2$. Let ϕ be convex.

$$\mathbb{E}[\phi(X_1)] = \mathbb{E}[\phi(|\sigma_1| Z)], \qquad Z \sim N(0,1).$$

Indeed,

$$X_1 \sim N(0, \sigma_1^2) \sim |\sigma_1|\, N(0,1).$$

The same relation trivially holds for X_2.

We consider the function

$$u \mapsto \mathbb{E}[\phi(uZ)] := f(u).$$

It is even since Z and $-Z$ have the same law, and hence

$$\mathbb{E}[\phi(uZ)] = \mathbb{E}[\phi(-uZ)].$$

It is also convex as

$$\begin{aligned} \mathbb{E}[\phi((\lambda u + (1-\lambda)v)Z)] &= \mathbb{E}[\phi(\lambda uZ + (1-\lambda)vZ)] \\ &\leq \lambda \mathbb{E}[\phi(uZ)] + (1-\lambda)\mathbb{E}[\phi(vZ)], \end{aligned}$$

where the last inequality comes from the convexity of ϕ.

Now, as f is convex and even,

$$\begin{aligned} f(0) &= f\big(\tfrac{u}{2} + -\tfrac{u}{2}\big) \\ &\leq \tfrac{f(u)}{2} + \tfrac{f(-u)}{2} = f(u), \qquad \forall u. \end{aligned}$$

Hence $u = 0$ is a minimum.
By convexity applied to

$$a = \tfrac{a}{b} b + 0\big(1 - \tfrac{a}{b}\big)\,,$$

we get

$$\begin{aligned} f(a) &\leq \tfrac{a}{b} f(b) + \big(1 - \tfrac{a}{b}\big)\, f(0) \\ &\leq \tfrac{a}{b} f(b) + \big(1 - \tfrac{a}{b}\big)\, f(a), \end{aligned}$$

which implies $f(a) \leq f(b)$ if $0 \leq a \leq b$.

Hence f is increasing on $[0, \infty)$ and

$$\mathbb{E}[\phi(X_1)] = \mathbb{E}[\phi(|\sigma_1|Z)] \leq \mathbb{E}[\phi(|\sigma_2|Z)] = \mathbb{E}[\phi(X_2)].$$

Therefore

$$N(0, \sigma_1^2) \leq_{cvx} N(0, \sigma_2^2).$$

An example of a martingale is obtained with

$$\sigma_1 = 1, \qquad \sigma_2 = 2,$$

so that

$$X_1 \sim N(0, 1), \qquad X_2 \sim N(0, 4).$$

Take

$$M_1 = X_1, \qquad M_2 = X_1 + \sqrt{3}\, Z,$$

where $Z \sim N(0, 1)$ is independent of X_1.

Then

$$M_1 \sim X_1, \qquad M_2 \sim N(0, 4) \sim X_2.$$

Moreover,

$$\begin{aligned} \mathbb{E}[M_2 \mid M_1] &= \mathbb{E}[X_1 + \sqrt{3} Z \mid X_1] \\ &= X_1 + \sqrt{3}\mathbb{E}[Z \mid X_1] \\ &= X_1 + \sqrt{3}\mathbb{E}[Z] \\ &= X_1. \end{aligned}$$

Hence (M_1, M_2) is a martingale.

An example of measures not increasing in convex order is the following.

Let μ_1 be such that

$$\mu_1(0) = 1,$$

and μ_2 such that

$$\mu_2(0) = \tfrac{1}{2}, \qquad \mu_2(2) = \tfrac{1}{2}.$$

Let $X_1 \sim \mu_1$ and $X_2 \sim \mu_2$.

Assume μ_1 and μ_2 are increasing in convex order. For $\phi(x) = x$ (which is convex) we must have

$$\mathbb{E}[X_2] \geq \mathbb{E}[X_1].$$

But ϕ is also concave, hence $-\phi$ is convex, so we must also have

$$\mathbb{E}[X_2] \leq \mathbb{E}[X_1].$$

Therefore

$$\mathbb{E}[X_2] = \mathbb{E}[X_1].$$

However,

$$\mathbb{E}[X_2] = 1, \qquad \mathbb{E}[X_1] = 0,$$

which is absurd. Thus μ_1 and μ_2 are not increasing in convex order.

Suppose we try to build a martingale (M_1, M_2) such that

$$M_1 \sim X_1, \qquad M_2 \sim X_2.$$

If it existed, we would have

$$\mathbb{E}[M_2 \mid M_1] = \mathbb{E}[X_2 \mid X_1].$$

But $M_1 = X_1 = 0$ a.s., hence

$$\mathbb{E}[X_2 \mid X_1] = \mathbb{E}[X_2] = 1 \neq 0 = M_1.$$

Hence no martingale (M_1, M_2) can have (X_1, X_2) as marginals, which confirms Kellerer's theorem in this example.

Now to be able to apply Kellerer's theorem, we will try to find some sufficient conditions on the call prices such that the risk neutral densities are increasing in the convex order.

Consider option prices $C_{i,j}$ over a set of strikes K_i, and maturities T_j. We assume the strikes are starting at 0, increasing in i and go to $+\infty$ as i goes to $+\infty$. We impose the following constraints on the call prices:

$lim_{i\to\infty} C_{i,j} = 0 \quad \forall j$. Indeed, As the strikes go to ∞, the call should be worthless as it is impossible to predict the stock price.

$C_{0,j} = S_0$: We get the asset for free at T_j, hence the price must be the current value of the asset.

$C_{i,0} = (S_0 - K_i)^+ \quad \forall i$. Indeed, if the maturity is 0, the call should be worth its payoff.

Now we define:
$Q_{i,j} := \frac{C_{i-1,j} - C_{i,j}}{K_i - K_{i-1}}, Q_{0,j} = 1$.

We impose that $Q_{i,j} \in [0,1]$. Indeed, $Q_{i,j}$ represents the normalized price of a vertical call spread, consisting of buying a call with strike K_{i-1} and selling a call with strike K_i. Let us consider the payoff of such a strategy at maturity T:

$$\text{Payoff}_T = (S_T - K_{i-1})^+ - (S_T - K_i)^+.$$

Using the monotonicity of the function $x \mapsto (x-K)^+$, we can bound this payoff:

- If $S_T \leq K_{i-1}$, the payoff is 0.
- If $K_{i-1} < S_T < K_i$, the payoff is $S_T - K_{i-1} < K_i - K_{i-1}$.
- If $S_T \geq K_i$, the payoff is $(S_T - K_{i-1}) - (S_T - K_i) = K_i - K_{i-1}$.

Thus, the payoff is strictly bounded by the difference in strikes:

$$0 \leq \text{Payoff}_T \leq K_i - K_{i-1}.$$

By the no-arbitrage principle, the current price of this strategy must satisfy the same bounds. Dividing by the spread width $K_i - K_{i-1}$, we conclude that the ratio $Q_{i,j}$ must lie in $[0,1]$.

We also impose $Q_{i,j} \geq Q_{i+1,j}$. This comes from the fact that if we define $BS_{i,j} = Q_{i,j} - Q_{i+1,j}$ (called a butterlfy spread), its payoff is non-negative, hence its price must be non-negative.

We then define: $q_{i,j} = Q_{i,j} - Q_{i+1,j}$. We have: $\sum_{i=0}^{\infty} q_{i,j} = lim_{i \to} \infty (Q_{0,j} - Q_{i,j}) = 1$. And to each maturity, we associate the risk neutral measure defined by: $F_j(K) = \sum_{K_l \leq K} q_{l,j}$. Hence: $F_j(k)$ is a probability measure.

To formally prove that the risk-neutral densities are increasing in the convex order with respect to maturity (i.e., satisfying the Kellerer theorem condition), we invoke the Carr-Madan spanning formula.

Recall that any twice differentiable function ϕ can be replicated using a continuum of Call options. Assuming S_t is a martingale and S_0 is fixed, for any convex function ϕ, we have:

$$\phi(S_T) = \phi(S_0) + \phi'(S_0)(S_T - S_0) + \int_0^{\infty} \phi''(K)(S_T - K)^+ \, dK.$$

Taking the expectation under the risk-neutral measure (where $\mathbb{E}[S_T] = S_0$) and using Fubini, we get:

$$\mathbb{E}[\phi(S_T)] = \phi(S_0) + \int_0^{\infty} \phi''(K) C(K,T) \, dK.$$

Now, consider two maturities $T_1 < T_2$. The condition for convex order $S_{T_1} \preceq_{cx} S_{T_2}$ is defined by:

$$\mathbb{E}[\phi(S_{T_1})] \leq \mathbb{E}[\phi(S_{T_2})], \quad \forall \phi \text{ convex}.$$

Using the expansion above, the difference in expectations is:

$$\mathbb{E}[\phi(S_{T_2})] - \mathbb{E}[\phi(S_{T_1})] = \int_0^\infty \phi''(K) \left(C(K, T_2) - C(K, T_1)\right) dK.$$

Since ϕ is convex, $\phi''(K) \geq 0$ for all K. To guarantee that this integral is non-negative for *any* convex function ϕ, a necessary and sufficient condition is:

$$C(K, T_2) \geq C(K, T_1), \quad \forall K.$$

To formalize the no-arbitrage requirements, we bridge the Carr-Madan formula with the result of Breeden and Litzenberger (proven in [7]). They demonstrated that the risk-neutral probability density function $f_{\mathbb{Q}}(K)$ is proportional to the second partial derivative of the Call price with respect to the strike:

$$f_{\mathbb{Q}}(K) = e^{rT} \frac{\partial^2 C}{\partial K^2}. \tag{3}$$

From this perspective, the requirement of absence of static arbitrage is equivalent to the non-negativity of this density. Any violation of the price convexity ($\frac{\partial^2 C}{\partial K^2} < 0$) would imply a negative probability for certain states of the world, thereby creating a "butterfly" arbitrage opportunity.

In continuous setting, our linear constraints on call option prices to guarantee absence of static arbitrage become:

$$\frac{\partial \check{c}(T,k)}{\partial k} \in [-1, 0], \frac{\partial^2 \check{c}(T,k)}{\partial^2 k} \geq 0, \frac{\partial \check{c}(T,k)}{\partial T} \geq 0$$

.

Hence if we define for D a compact set:

$$S(D) = s(x,y) \in C^{1,2}(D) : 0 \leq s \leq 1, \frac{\partial s}{\partial x} \geq 0, -1 \leq \frac{\partial s}{\partial y} \leq 0, \frac{\partial^2 s}{\partial^2 y} \geq 0$$

.

Then if $\forall D, \check{c}_{|D} \in S(D)$, there can be no static arbitrage.
For us:

$$D = \check{R}_{liq} := \{(\tau, k) : (\tau, log(k)) \in R_{liq}\}.$$

We also define the discrete grid:

$$\check{L}_{liq} = \{(\tau_i, k^i_j) : (\tau_i, log(k^i_j) \in L_{liq}\}$$

We then proceed in two steps:

first, we want to make sure that given the prices in $\check{L}_{liq}$, we can find a function $s \in S(\check{R}_{liq})$ such that for all $(\tau, k) \in \check{L}_{liq}$, $s(\tau, k) = \check{c}(\tau, k)$.

Then, given the prices in $\check{L}_{liq}$, we apply a smooth interpolation method which preserves the monotonicity (on τ and k) and the convexity (on k) of the surface.

To deal with the first step:

It has been proven in [5] that in our context, the no static arbitrage constraints can be summarized as: $Ac \geq \hat{b}$, where $c = [c_1, ...c_N]^T$, are the normalised call prices. $A \in \mathbb{R}^{R \times N}, \hat{b} \in \mathbb{R}^R$. In our model: $c_t = G_0 + G^T \xi_t, G_{i,j} = G_i(\tau_j, m_j)$. Hence:

$$L_{\text{liq}} \quad \text{is free of static arbitrage} \iff \xi_t \quad \text{satisfies} : AG^T \xi_t \geq \hat{b} - AG_0^T, \forall t \in [0, T^\star - \tau_{sup}]$$

That allows us to have conditions on the hidden factors directly for absence of static arbitrage.

4.2 Dynamic arbitrage

A dynamic arbitrage opportunity is any arbitrage opportunity which is not static.

Example.
Consider the Black–Scholes model, where the dynamics of the underlying asset are given by

$$dS_t = rS_t\,dt + \sigma S_t\,dW_t.$$

Let C be the quoted price of a call such that $C > C^{BS}$, where C^{BS} is the theoretical price given by the Black–Scholes model for this call. It is then well known that a replicating strategy can be found for the call, which consists of holding $\frac{\partial C_t}{\partial S}$ units of the asset at each time t.

The arbitrage then consists in selling the call (hence making a profit of C), and then replicating the call using the strategy in order to deliver the payoff to the option buyer. However, the price of this strategy is precisely the theoretical price of the call, namely C^{BS}. Our profit at maturity is therefore

$$C - C^{BS} > 0.$$

Note: As dynamic arbitrage opportunities usually depend on the model used, they are often referred to as *model-based arbitrage*.

Suppose we model $N = |\mathcal{L}_{\text{liq}}|$ options, with prices given in (τ, m)-coordinates.

It has been proved in the original paper ([8]) that absence of dynamic arbitrage is equivalent to the existence of ψ_t such that

$$\mathbf{G}^\top \sigma_t \psi_t = \mathbf{G}^\top \mu_t - z_t. \qquad (\star)$$

where

$$z_t = \left(-\frac{\partial}{\partial \tau} - \frac{1}{2}\gamma_t^2 \frac{\partial}{\partial m} + \frac{1}{2}\gamma_t^2 \frac{\partial^2}{\partial m^2} \right) \begin{bmatrix} \tilde{c}_t(\tau_1, m_1) \\ \vdots \\ \tilde{c}_t(\tau_N, m_N) \end{bmatrix}.$$

Here, we define $\mathbf{G}$ as the $d \times N$ matrix with i-th row $\mathbf{G}_i$, where

$$\mathbf{G}_i = (G_{ij})_{j=1}^N \in \mathbb{R}^N, \quad i = 0, \dots, d,$$

and

$$G_{ij} = G_i(\tau_j, m_j).$$

We call $\mathbf{G}^\top$ a *price basis* and each of its columns a *price basis vector*.

If $z_t \in \mathrm{col}(\mathbf{G}^\top)$, i.e. if there exists β_t such that

$$z_t = \mathbf{G}^\top \beta_t,$$

then condition $(\star)$ becomes

$$\mathbf{G}^\top \sigma_t \psi_t = \mathbf{G}^\top (\mu_t - \beta_t).$$

In particular, if

$$\sigma_t \psi_t = \mu_t - \beta_t,$$

then this implies the existence of ψ_t satisfying the condition. Assuming that σ_t is invertible, we obtain

$$\psi_t = \sigma_t^{-1}(\mu_t - \beta_t),$$

which is a solution to $(\star)$.

Hence, the choice of $\mathbf{G}$ is more important than the choice of μ_t and σ_t to guarantee absence of arbitrage. However, $\mathrm{col}(\mathbf{G}^\top)$ is a d-dimensional space. As $N \gg d$, z_t has in practice no chance to belong to a d-dimensional space, which has Lebesgue measure 0 in $\mathbb{R}^N$.

We then want to minimize

$$\min_{\mathbf{G},\, \phi} \left\| z_t - \mathbf{G}^\top \phi \right\|^2$$

such that $\mathbf{G}^\top \phi$ is of rank d.

As the function $\mathbf{G}$ is the same for every time

$$0 = t_0 < t_1 < \cdots < t_L,$$

we regroup the observations in

$$Z \in \mathbb{R}^{(L+1)\times N},$$

and the optimization problem now reads

$$\min_{\mathbf{G},\,\phi} \|Z - \phi\mathbf{G}\|_F^2 \quad \text{such that} \quad \operatorname{rank}(\phi\mathbf{G}) = d.$$

This minimization problem will be solved while constructing the factors.

5 Decoding the factors

5.1 Setting and objectives

We define matrices

$$C = (C_{l,j}) \in \mathbb{R}^{(L+1)\times N} \quad \text{and} \quad \Xi = (\xi_{l,i}) \in \mathbb{R}^{(L+1)\times d},$$

by

$$C_{l,j} = \tilde{c}_{t_{l-1}}(\tau_j, m_j), \qquad \xi_{l,i} = \xi_{i,t_{l-1}},$$

respectively.

We represent the observed data C with residuals Y:

$$C = \mathbf{1}_{L+1} G_0^\top + \Xi G^\top + Y. \tag{4}$$

Here, $\mathbf{1}_{L+1}$ is the $(L+1)$-vector of ones. We call $\hat{C}$ the reconstructed prices, and the Frobenius norm $\|Y\|_F$ the reconstruction error.

For some fixed $d < N$, we want to achieve the three following goals:

- minimize static arbitrage,
- minimize dynamic arbitrage,
- maximize statistical accuracy, i.e., make $\|Y\|_F$ as small as possible.

To deal with the second goal, by [9], we know that the PCA method gives the best result among linear dimension-reduction techniques. Indeed, for $X \in \mathbb{R}^{n\times p}$ a centered data matrix, we want to find $a \in \mathbb{R}^p$ such that

$$a = \arg\max_a \quad a^\top S a, \qquad \text{subject to } a^\top a = 1, \tag{5}$$

where S is the empirical covariance matrix.

To solve this, we set the singular value decomposition

$$X = ULA^\top,$$

where the columns of A are the eigenvectors of $X^\top X$, which represent the directions of maximal variance, and the columns of UL are called the principal components of X.

The key result is that:

$$X_q := U_q L_q A_q^\top = \arg\min_{\operatorname{rank}(B)=q} \|X - B\|_F^2. \tag{6}$$

Hence, PCA provides the best q-dimensional approximation of the data.

In our setting, this translates into

$$\min_{\substack{G,\phi \\ \operatorname{rank}(\phi G)=d}} \quad \|Z - \phi G\|_F^2 = \left\|Z - U_d L_d A_d^\top\right\|_F^2. \tag{7}$$

Hence,

$$G^\top = U_d L_d,$$

and the rows of G are the principal components of Z.

We can now derive the following algorithm.

5.2 Algorithm

Algorithm 1 Decoding the factor representation

Input: Matrix of price data $C \in \mathbb{R}^{(L+1)\times N}$; number of factors d; number of statistical accuracy factors d_{st}; number of dynamic arbitrage factors d_{da}; number of static arbitrage factors d_{sa}; data for z_t , denoted as $Z \in \mathbb{R}^{(L+1)\times N}$; constraint matrices A, b.

Output: Factor data $\Xi \in \mathbb{R}^{(L+1)\times d}$; price basis vectors $G_i \in \mathbb{R}^N$ for $i = 0, \ldots, d$.

1. Let $G_{0,j} = \frac{1}{L+1}\sum_{l=1}^{L+1} C_{l,j}$ for $j = 1, \ldots, N$

`/* Construct factors to minimize dynamic arbitrage */`

2. Compute the residual data $R^0 = C - \mathbf{1}_{L+1}G_0^\top$ 3. Compute the principal component decomposition of Z and assign, for $i = 1, \ldots, d_{da}$ and $l = 1, \ldots, L+1$:

- $G_i = i$-th principal component of Z
- $\xi_{l,i} = \langle (R^0)_l, G_i \rangle$ where $(R^0)_l$ is the l-th row of R^0

`/* Construct factors to maximize statistical accuracy */`

4. Compute the residual data $R_{dda} = R^0 - \sum_{k=1}^{d_{da}} \xi_k G_k^\top$, where $\xi_k = [\xi_{1,k}, \ldots, \xi_{L+1,k}]^\top$ 5. Compute the principal component decomposition of R_{dda} and assign, for $i = d_{da}+1, \ldots, d_{da}+d_{st}$:

- $G_i = (i - d_{da})$-th principal component of R_{dda}
- $\xi_{l,i} = (R_{dda})_l G_i$

`/* Construct factors to minimize static arbitrage */`

for $i = d_{da} + d_{st}$ **to** $d - 1$ **do**

6. Compute the residual data $R^i = R^0 - \sum_{k=1}^{i} \xi_k G_k^\top$ 7. Compute the covariance of the residual data $\Sigma_i = (R^i)^\top R^i$ 8. Eigen-decompose $\Sigma_i = Q_i \Lambda_i Q_i^{-1}$ 9. For some $w \in \mathbb{R}^d$ such that $\|w\|_2 = 1$, define $q_i(w) = Q_i w$ and $s_i(w) = R^i q_i(w)$ 10. Solve the following minimization problem:

$$\mathbf{w}^* = \arg\min_{\|\mathbf{w}\|_2=1} \left(-\sum_{l=1}^{L+1} \mathbf{1}\left\{ \langle \mathbf{A}\mathbf{q}_i(\mathbf{w}), \mathbf{s}_i(\mathbf{w}) \rangle \geq b - A\left(\sum_{k=1}^{i-1} \xi_{lk} G_k \right) \right\} + \lambda \left\| \mathbf{R}_i - \mathbf{s}_i(\mathbf{w}) \otimes \mathbf{q}_i(\mathbf{w}) \right\|_2 \right),$$

where we require $\lambda < min(1, \frac{1}{\|R_i\|})$ 11. Let $G_{i+1} = q_i(w^*)$ and $\xi_{i+1} = s_i(w^*)$

end

return $\xi, \{G_i\}_{i=0}^d$

The construction of this algorithm can be justified as follows:

For minimizing dynamic arbitrage:

$G_{0,j}$ is the average price of the call j over time. We define R^0 to center the prices with respect to the mean of each call price over time.

We define $\xi_{l,i}$ as $\langle (R^0)_l, G_i \rangle$, because we want to minimize $\|R_{dda}\|_2^2 = \|R^0 - \sum_{k=1}^{d_{da}} \xi_k G_k^\top \|_2^2$. By applying first order conditions, we want to solve:

$$-2G^T(R_0)l + 2G^T G\xi_l = 0, \quad \text{which leads to} \quad \xi_l = (G^T G)^{-1} G^T (R_0)_l$$

Hence if G is orthonormal, we find:

$$\xi_l = G^T (R_0)_l, \quad \text{hence} \quad \xi_{l,i} = \langle (R_0)_l, G_i \rangle$$

For maximizing statistical accuracy:

The new residual reads:

$$R_{dda} = R^0 - \sum_{k=1}^{d_{da}} \xi_k G_k^\top$$

By the same argument as before, we find the expression for $\xi_{l,i}$. Now to choose G_i, we want to find G_i such that it explains the maximum amount of residual variance. Hence we want to find:

$$G_i = argmax \quad Var(R_{dda} v) \quad , \|v\|_2 = 1$$

But we know that this optimization problem is solved ny taking G_i the i-th principal component of R_{dda}.

To minimize static arbitrage:

We have here two joint goals: applying the static arbitrage constraints, and reducing the residual part still not explained.
We iterate over i for i starting from $d_{da} + d_{st}$ to d-1 the following steps:
The covariance matrix of the residuals is given by:

$\Sigma_{ij,k} = \sum_{l=1}^{L+1} R_i(l,j) R_i(l,k)$

As Σ_i is symmetric, by the spectral theorem, we can decompose:

$$\Sigma_i = Q_i \Lambda_i Q_i^{-1} \quad \text{such that} \quad Q_i^T Q_i = Id$$

We want to find the unitary direction q that maximizes the variance of $R_i q$. We have:

$$Var(R_i q) = \frac{1}{L+1} \sum_{l=1}^{L+1} s_l^2 \quad \propto \quad \|R_i q\|_2^2 = q^T \Sigma_i q$$

Hence we need to maximize the Rayleigh quotient: $q^T \Sigma_i q$. But a classical result in linear algebra tells us that this quotient is maximized at the eigenvector associated to the largest eigenvalue of Σ_i, hence the columns of Q_i. So, we want to find w such that $\|w\| = 1, \|R_i w\|_2^2$ is maximal, and set $G_{i+1} = Q_i w$, $\xi_{i+1} = R_i Q_i w$. We search for $Q_i w$ (and not just any w) to limit the search into directions where there is residual variance, as we know that the column space of Q_i is generated by exactly these directions. Hence, the maximization of the variance does not appear explicitly in the minimization problem, but rather in the geometrical structure of the problem: we restrict the search in the column space of Q.

Then, we also want to minimize static arbitrage, which is done at the point 10 in the algorithm, where we are just counting the times when the static arbitrage constraints are borken. We add a regularization term to not overly fit the static arbitrage constraints without caring about how much residual we introduce, and also for ensuring uniqueness in the case where several w produce the same number of arbitrage free samples.

6 Neural SDE model

6.1 A constrained supervised learning problem

We want to estimate $\alpha, \mu, \gamma, \sigma$ in:

$$\begin{cases} dS_t = (\alpha(\tilde{\xi}_t) - q_t)S_t\, dt + \gamma(\tilde{\xi}_t)S_t\, dW_{0,t} \\ d\xi_t = \mu(\tilde{\xi}_t)\, dt + \sigma(\tilde{\xi}_t)\, dW_t \end{cases}$$

where $\tilde{\xi} = (S, \xi)$

As discussed in the previous sections, ξ should be constrained in the following set: $AG^T\xi_t \geq \hat{b} - AG_0^T$

As the process $\tilde{\xi}$ is a strong solution to a SDE where the coefficients are only functions of the current state and of time, it is strong Markov.
To avoid heavy notations, we will deal with this problem considering a generic markovian diffusion Y solution of:

$$dY_t = \mu(Y_t)dt + \sigma(Y_t)dW_t$$

We assume μ and σ are continuous and satisfy:
$|\mu(x) - \mu(y)| \leq K|x - y|$,
as well as:
$|\mu(x)| \leq K(1 + |x|)$, and same for σ.

This ensures that the considered SDE admits a unique strong solution.

We consider the next problem: we are given discrete observations of Y and want to estimate μ and σ such that $Y_t \in P := \{y \in \mathbb{R}^d : Vy \geq b\}$ for all $t \geq 0, \mathbb{P} - as$.
We set: $\tilde{v_i} = \frac{v_i}{\|v_i\|}$, $\tilde{b_i} = \frac{b_i}{\|b_i\|}$. Then: $Vy \geq b \iff \tilde{V}y \geq \tilde{b}$ and $\tilde{V}$ has unitary rows. Hence, we can assume without loss of generality that V has unitary rows. For each k, the diffusion process Y should not enter the domain

$$P_k^c := \{y \in \mathbb{R}^d : v_k^\top y < b_k\}$$

at any time $t > 0$, $\mathbb{P}$-a.s.

Now we define:

$$P^c := \bigcup_k P_k^c = \mathbb{R}^d \setminus P,$$

and we want to have:

$$\mathbb{P}(\exists\, t > 0 \text{ such that } Y_t \in P^c \,|\, Y_0 \in P) = 0.$$

In plain words, we want to make sure that the process stays in P if it starts in P. Thanks to [10], we have a result for this:

Theorem: Let $a = \sigma\sigma^T$. If μ and σ satisfy the conditions for uniqueness and existence of a strong solution and if:

$$\sum_{i=1}^{d} \mu_i \nu_i \; + \; \frac{1}{2} \sum_{i,j=1}^{d} a_{ij} \, \frac{\partial^2 \rho_k}{\partial y_i \partial y_j} \geq 0 \quad \text{on } \partial P_k.$$

where $\rho_k(y) := dist(y, P_k^c)$, and ν is the outward normal to ∂P_k (a vector towards the interior of P). Then:

$$\mathbb{P}(\exists\, t > 0 \text{ such that } Y_t \in P^c \,|\, Y_0 \in P) = 0.$$

But then we note that v_k is normal to ∂P_k as $\forall y_1, y_2 \in \partial P_k : v_k^T (y_1 - y_2) = b_k - b_k = 0$.

Hence:
$\nu = v_k$.

Moreover, we have: $\rho_k(y) = v_k^T y - b_k$ for every $y \in P_k$. Hence: $\nabla \rho_k = v_k$ and: $\nu = \nabla \rho_k$.

The conditions then become:

$$v_k^\top a(y)\, v_k = 0 \quad \text{if } v_k^\top y = b_k, \tag{8}$$
$$v_k^\top \mu(y) \geq 0 \quad \text{if } v_k^\top y = b_k. \tag{9}$$

We can now formulate the problem as follows:
We observe $y_{0:T}$, the discrete sample path at times $0 = t_0 < t_1 < .... < t_L = T$, and we want to solve:

$$\max_{\mu,\sigma} \; p(y_{0:T}; \mu, \sigma) \quad \text{subject to (8) and (9)}$$

Assuming a uniform time grid mesh, we set: $\Delta_t = \frac{T}{L}$.
Thanks to the Euler-Maruyama scheme, we get the following approximation:

$$Y_{t_{i+1}} - Y_{t_i} = \mu(Y_{t_i})\; \Delta t + \sigma(Y_{t_i})\; \big(W_{t_{i+1}} - W_{t_i}\big).$$

Hence:

$$\begin{aligned} Y_{t_{i+1}} \mid Y_{t_i} &\sim N\big(Y_{t_i} + \mu(i)\Delta_t,\, \sigma(i)\Delta_t \sigma(i)^T\big) \\ &= N\big(Y_{t_i} + \mu(i)\Delta_t,\, \Delta_t a(i)\big). \end{aligned}$$

And so we have:

$$p(Y_{t_{i+1}} \mid Y_{t_i}; \mu, \sigma) = \frac{1}{\sqrt{(2\pi)^d |a_i|}} \exp\left(-\frac{(Y_{t_{i+1}} - Y_{t_i} - \mu_i \Delta_t)^T (\Delta_t a(i)^{-1})(Y_{t_{i+1}} - Y_{t_i} - \mu_i \Delta_t)}{2} \right)$$
$$= \frac{1}{\sqrt{(2\pi)^d |a_i|}} \exp\left(-\frac{\|Y_{t_{i+1}} - Y_{t_i} - \mu_i \Delta_t\|^2_{a(i)}}{2\Delta_t} \right),$$

where

$$\|u\|_a^2 := (u,u)_a \qquad \text{and} \qquad (u,v)_a := u^T a^{-1} v.$$

Then by the product rule, we have:

$$p(y_{0:T} \mid y_0; \mu, \sigma) = \prod_{i=0}^{L-1} p(y_{t_{i+1}} \mid y_{t_i}, \ldots, y_{t_0}; \mu, \sigma).$$

As Y is a Markov process, we then have:

$$p(y_{0:T} \mid y_0; \mu, \sigma) = \prod_{i=0}^{L-1} p(y_{t_{i+1}} \mid y_{t_i}; \mu, \sigma).$$

Moreover:

$$\begin{aligned}
\|Y_{t_{i+1}} - Y_{t_i} - \mu_i \Delta_t\|^2_{a(i)} &= (Y_{t_{i+1}} - Y_{t_i} - \mu_i \Delta_t) a(i) (Y_{t_{i+1}} - Y_{t_i} - \mu_i \Delta_t) \\
&= (Y_{t_{i+1}} - Y_{t_i})^T a(i) (Y_{t_{i+1}} - Y_{t_i}) \\
&\quad - (Y_{t_{i+1}} - Y_{t_i})^T a(i) \mu_i \Delta_t \\
&\quad - (\mu_i \Delta_t)^T a(i) (Y_{t_{i+1}} - Y_{t_i}) \\
&\quad + (\mu_i \Delta_t)^T a(i) \mu_i \Delta_t \\
&= \|Y_{t_{i+1}} - Y_{t_i}\|^2_{a(i)} + (\Delta_t)^2 \|\mu(i)\|^2_{a(i)} - 2\Delta_t (\mu_i, y_{t_{i+1}} - y_{t_i})_{a(i)}.
\end{aligned}$$

Hence:

$$\log p(y_{0:T} \mid y_{t_0}, \mu, \sigma) \propto -\frac{1}{2} \sum_{i=0}^{L-1} \log |a_{(i)}| - \frac{1}{2} \sum_{i=0}^{L-1} \frac{1}{\Delta t} \|Y_{t_{i+1}} - Y_{t_i}\|^2_{a(i)} + \|\mu(i)\|^2_{a(i)} \Delta_t - 2(\mu_i, y_{t_{i+1}} - y_{t_i})_{a(i)}$$

And we want to find μ and σ that maximize the right hand side. We can view this as a ML supervised learning task where we learn the outputs y_i thanks to the inputs x_i by setting:

$$x_i := y_{t_i}, y_i := y_{t_{i+1}} - y_{t_i}$$

We can then take $\{(x_i, y_i)\}$ as the training set. We define the following loss:

$$l(x_i, y_i) = log(|a_{(i)}|) + \frac{1}{\Delta t} \|y_i\|^2_{a(i)} + \|\mu(i)\|^2_{a(i)} \Delta_t - 2(\mu_i, y_i)_{a(i)}$$

This is mathematically equivalent to predicting just y_{i+1}, but it is preferable to write it as an increment since we have:

$\Delta Y_{t_i} = \mu(Y_{t_i})\Delta_t + \sigma(Y_{t_i})\Delta_{W_i}$

which can then be read as: Target = Input + Input*Noise

We want to learn μ and σ by using neural networks.

6.2 Modelling the constraints

μ and σ must satisfy:

$$v_k^\top a(y)\, v_k = 0 \quad \text{if } v_k^\top y = b_k, \tag{10}$$

$$v_k^\top \mu(y) \geq 0 \quad \text{if } v_k^\top y = b_k. \tag{11}$$

which in our case, amounts to:
$AG^T\xi_t \geq \hat{b} - AG_0^T$

We do so by setting: $\mu = G_\mu[\hat{\mu}], \sigma = G_\sigma[\hat{\sigma}]$, where G_μ and G_σ are such that the constraints are verified, and $\hat{\mu}$ and $\hat{\sigma}$ are the outputs of the neural networks. we then want to build such G_μ and G_σ.

For G_σ: σ is constrained only if $\rho_k(y) = v_k^T y - b_k = 0$. Then we must have: $v_k^T \sigma\sigma^T v_k = 0$. But $v_k^T \sigma\sigma^T v_k = (v_k^T \sigma)^2$. Hence we want G_σ such that for any lipschitz continuous function: $\hat{\sigma} : \mathbb{R}^d \to \mathbb{R}^\times$, we have that $\sigma = G_\sigma[\hat{\sigma}] \in ker(vk^T)$ if $p_k(y) = 0$.

For that, we use the following algorithm.

Algorithm 2 Constructing diffusion shrinking transformation matrix

Input : A non-empty d-polytope

$$P = \{x \in \mathbb{R}^d : Vx \leq b\}, \quad V \in \mathbb{R}^{R\times d},\ b \in \mathbb{R}^d;$$

a Lipschitz continuous and monotonically increasing function

$$h : \mathbb{R}_{\geq 0} \to \mathbb{R}\,[0, 1]$$

satisfying $h(0) = 0$ and $h(x) \to 1$ for $x \to \infty$; some fixed $y \in \mathbb{R}^d$.

Output: The shrinking transformation $P(y) \in \mathbb{R}^{d\times d}$.

/ In our experiments, we use the simple example $h(x) = 1 - \frac{1}{1+x}$. */*

for $k = 1, 2, \ldots, R$ **do**

Construct the normalised distances

$$\xi_k = h\big(\rho_k(y)\big),$$

where ρ_k is the Euclidean distance to the k-th boundary of P.

Sort ξ_k in ascending order, and denote the corresponding permutation of the indices by π. That is,

$$\xi_{(k)} = \xi_{\pi(k)}.$$

Keep the first d minimal $\xi_{(k)}$, denoted by $\xi_{(k)},\ 1 \leq k \leq d$, and the associated hyperplane boundary coefficients $v_{(k)},\ 1 \leq k \leq d$.

Let $U = [v_{(1)}\ \cdots\ v_{(d)}]$ and assume it has full rank.

(To ensure U is of full rank, we can exclude from the ordering any k such that $v_{(k)} \in \operatorname{span}\{v_{(1)}, \ldots, v_{(k-1)}\}$.)

/ The set $\{v_{(k)}\}_k$ represents the d directions on which the projected diffusion components most need to vanish. Therefore, we consider shrinking the diffusion along the orthogonal basis that is formed by these directions according to the order and scale of the corresponding distances $\{\xi_{(k)}\}_k$. */*

Let $q_1 = v_{(1)}$.

for $k = 2, \ldots, d$ **do**

Following the Gram–Schmidt process, let

$$q_k = (v_{(k)} - \sum_{j=1}^{k-1} \langle q_j, v_{(k)}\rangle q_j) \frac{1}{\sqrt{1 - \sum_{j=1}^{k-1} \langle q_j, v_{(k)}\rangle^2)}}$$

Then q_k is the unit component of $v_{(k)}$ that is orthogonal to $\operatorname{span}\{v_{(1)}, \ldots, v_{(k-1)}\}$.

Let $Q = [q_1\ \cdots\ q_d]$ be the resulting change-of-basis matrix.

Define

$$P(y) = \operatorname{diag}\big(\sqrt{\xi_{(1)}}, \ldots, \sqrt{\xi_{(d)}}\big)\, Q^\top$$

Explanation of the algorithm:

The algorithm is pretty straight-forward, except for the definition of U. We define U as $U = (V_{(1)}, ..., V_{(d)})$ because these are the d directions on which the projected diffusion components most need to vanish. Given y, we must see which faces (k) are close to the polytope to see if $p_k(y)$ can become 0 in a short amount of time. Hence we take the d such closest faces and identify them with the normals which are the directions in which Y could exit the polytope in a short time span Δ_t. By Gram-Schmidt, we then orthonormalize $V_{(1)}, ...V_{(d)}$ into $[q_1, ..., q_d] = Q$. And we define $P(y) = \operatorname{diag}\big(\sqrt{\xi_{(1)}}, \ldots, \sqrt{\xi_{(d)}}\big)\, Q^\top$.

We then have the following result, which tells us that the operator constructed thanks to algorithm 2 is indeed constrained in the polytope.

Theorem:
Given the operator G defined by algorithm 2:

$$\text{If} \quad \rho_k(y) = 0, \quad \text{then} \quad G_\sigma[\hat{\sigma}] \in \ker(v_k^T)$$

for each k and any function $\sigma : \mathbb{R}^d \to \mathbb{R}^{d\times d}$.

Proof:
We assume $p_k(y) = 0$. Hence $\xi_{\pi(k)} = \xi_k = 0$.

Case 1: $\pi(k) \geq d$.

Then $\xi_{(j)} = 0 \quad \forall j \in \{1, \ldots, d\}$. Hence

$$v_k^\top a v_k = \sum_{i,j=1}^{d} a_{ij} \left(v_k^\top q_i\right) \left(v_k^\top q_j\right) \sqrt{\varepsilon_{(i)}\, \varepsilon_{(j)}} = 0.$$

Case 2: $1 \leq \pi(k) < d$.

Since

$$v_k = \sum_{j\leq k} (v_k^\top q_j)\, q_j,$$

we have $v_k \in \operatorname{span}(q_1, \ldots, q_{\pi(k)})$, which is orthogonal to q_j for all $j \geq \pi(k)$.

We then deduce

$$v_k^\top a v_k = \sum_{i,j=1}^{d} a_{ij} \left(v_k^\top q_i\right) \left(v_k^\top q_j\right) \sqrt{\varepsilon_{(i)}\, \varepsilon_{(j)}}.$$

But $v_k = v_{(\pi(k))}$, which is orthogonal to q_j for all $j > \pi(k)$. Hence

$$v_k^\top a v_k = \sum_{i,j=1}^{\pi(k)} a_{ij} \left(v_k^\top q_i\right) \left(v_k^\top q_j\right) \sqrt{\varepsilon_{(i)}\, \varepsilon_{(j)}} = 0,$$

since $\xi_{(i)} = 0 \quad \forall i \leq \pi(k)$.

The approach to constrain the drift process is similar and given in the original paper.

6.3 Example of a diffusion constrained in a polytope

We compare the general Neural SDE framework described in the original paper with the heuristic numerical scheme used in this simulation to enforce boundary constraints.

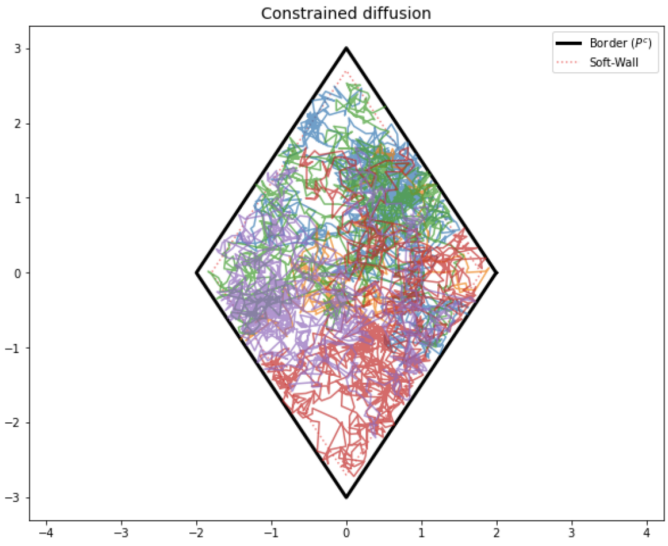


Figure 1: Trajectories of a diffusion process constrained in a polytope

Neural SDE: The Neural SDE model seeks to learn the drift $\mu(y)$ and diffusion $\sigma(y)$ via neural networks. To ensure the process Y_t remains within the no-arbitrage domain P, the network is trained to satisfy the Friedman-Pinsky conditions (8 and 9).

1. **Vanishing Volatility:** The variance must degenerate in the normal direction at the boundary ($\nu^\top a \nu = 0$).
2. **Inward Drift:** The drift must effectively push the process back into the domain ($\nu^\top \mu \geq 0$).

In this framework, the specific shape of these functions is not fixed *a priori* but learned from market data to maximize the likelihood of observed paths.

Approach used to generate the figure: To simulate valid trajectories without training a network, we constructed explicit functions that satisfy the theoretical conditions by design, using the distance to the boundary $d(y, \partial P)$:

- **Diffusion:** We model the volatility as $\sigma(y) \propto d(y)^{0.15}$. This power-law ensures the noise term vanishes exactly at the boundary while maintaining high volatility within the domain.

- **Drift:** We implement a repulsive potential barrier. The drift is zero in the center but becomes singular near the boundary ($\mu(y) \propto -1/d(y)$), acting as a "soft wall" that prevents the process from exiting P.

While our heuristic approach successfully enforces the domain invariance through hand-crafted barriers, the Neural SDE approach is superior for financial modeling as it can learn complex, data-driven boundary behaviors that reflect actual market dynamics.

7 Implementation of the model

We will use an Heston Stochastic Local Volatility to generate prices of european calls. We will split the generated data into a train and test set. We will then train the Neural SDE model on the train set and evaluate it on the test set using some evaluation metrics, as we will detailed below.

We simulate the trajectories of the underlying price and the volatility over 1 year (only one each), sampling every 10 minutes. The drift is set at 0, hence not from the risk neutral measure: however, it is not a problem from the arbitrage point of view in the neural SDE model since the no arbitrage constraints are imposed by the neural net directly. It is not a problem from the arbitrage point of view in Heston model neither, since we only want to generate realistic trajectories, not necessarily arbitrage free. $C_t(T, K)$ is then computed at each time t. The values of the price of the underlying asset and of the options at each time t are then given as inputs to the Neural SDE model.

Then, we have to decode the factors. For this we need to have access to z_t. To compute

$$z_t = \left(-\frac{\partial}{\partial \tau} - \frac{1}{2}\gamma_t^2 \frac{\partial}{\partial m} + \frac{1}{2}\gamma_t^2 \frac{\partial^2}{\partial m^2} \right) \begin{bmatrix} \tilde{c}_1(\tau_1, m_1) \\ \vdots \\ \tilde{c}_N(\tau_N, m_N) \end{bmatrix}$$

,

we need γ_t and $\tilde{c}_i(\tau_i, m_i) \quad \forall i \in 1, ..., N$. To achieve this, we compute γ_t using the method described in section 6.

We use this architecture for S:

$$\phi^{S,\theta} = \mathcal{F}_{d+1} \circ \mathcal{A}_{\text{ReLU}} \circ \mathcal{F}_{128} \circ \mathcal{A}_{\text{ReLU}} \circ \mathcal{F}_{128} \circ \mathcal{A}_{\text{ReLU}} \circ \mathcal{F}_{128}.$$

And this architecture for ξ:

$$\Phi_\theta = \mathcal{F}_{d+1} \circ \mathcal{A}_{\text{ReLU}} \circ \mathcal{F}_{256} \circ \mathcal{A}_{\text{ReLU}} \circ \mathcal{F}_{256} \circ \mathcal{A}_{\text{ReLU}} \circ F_{256}$$

The 50% smallest weights are then set at 0. The model takes as inputs $\tilde{\xi}^{(0)} = (S_0, \xi^{(0)})$.

7.1 Choice of the Heston–SLV Parameters and calibration

The synthetic data used in the numerical experiments are generated from a Heston-type Stochastic Local Volatility (Heston–SLV) model, which is treated as the ground-truth data-generating process. The parameters of this model are not estimated within the learning procedure, but are fixed *a priori* through calibration to real market data.

A Heston–SLV model is calibrated to Bloomberg OTC USD/BRL European option prices observed on October 28, 2008, using the QuantLib library. What

calibration means in this context:

- Selecting the model parameters so that the model-generated European option prices match the observed market prices at a fixed date.
- First, the Heston stochastic volatility parameters $(\kappa, \theta, \sigma, \rho, v_0)$ are fitted to the implied volatility surface.
- This step provides realistic stochastic volatility dynamics but does not perfectly reprice all vanilla options.
- Second, a leverage (local volatility) function $L_t(t, S_t)$ is calibrated, conditional on the Heston parameters, to exactly match vanilla option prices.
- The resulting Heston–SLV model is exactly calibrated to market vanillas, and is used as a realistic data-generating process.

Remark:
The values of q_t and r_t needed for the calibration phase are given by the Bloomberg data. However, once the calibration is complete—ensuring the parameters capture realistic volatility dynamics—the specific choice of q_t and r_t becomes irrelevant for the simulation. Indeed, in this context, the calibrated Heston–SLV model is not used to price assets under the original market constraints (and risk neutral measure), but rather serves as a ground truth generator. It provides a realistic and controlled environment to produce synthetic data against which the neural SDE model can be validated. Hence, we can fix them at 0 for simplicity.

7.2 Synthetic Data Generation Procedure

The synthetic dataset is generated from the following Heston–SLV model:

$$dS_t = (r_t - q_t)S_t\,dt + L_t(t, S_t)\sqrt{v_t}\,S_t\,dW_t^S, \tag{12}$$

$$dv_t = \kappa(\theta - v_t)\,dt + \sigma\sqrt{v_t}\,dW_t^v, \tag{13}$$

$$d\langle W^S, W^v\rangle_t = \rho\,dt, \tag{14}$$

where the parameters and leverage function are those obtained from the market calibration described above.

A discrete time grid with step size Δt and total length $(L+1)\Delta t$ is fixed. The grid is chosen to emulate moderately frequent observations (of the order of every ten minutes) over a one-year horizon.

A single trajectory of the state variables (S_t, v_t) on the interval $[0, T]$ is generated using an Euler–Maruyama discretization of the Heston–SLV stochastic differential equations.

For simplicity, the drift of S_t is chosen to be zero in the simulation. This choice is arbitrary and does not affect the option prices, which are recomputed

under the risk-neutral measure at each observation time. The simulated trajectory is selected so that S_t remains within the domain where the leverage function L is defined.

To generate option prices at time t, the leverage function is forward-translated according to

$$L_t(u, S) := L_0(u - t, S), \qquad u \in [t, T].$$

This ensures that, at each observation time, the model reprices vanilla options consistently with the initially calibrated surface.

7.3 Generation of option prices

For each observation time t along the simulated trajectory (S_t, v_t), prices of European call options are computed for a fixed grid of strikes and maturities.

These prices are obtained by pricing under the risk-neutral Heston–SLV model, started from the state (S_t, v_t), using the calibrated parameters and the translated leverage function. Each observation therefore takes the form

$$(S_t,\ \{C(t, K_i, T_j)\}_{i,j})\,.$$

We can then use Algorithm 1 to decode the factors. Recall the 3 objectives of this algorithm: maximize statistical accuracy, minimize static arbitrage, minimize dynamic arbitrage.

We will use one metric to evaluate each goal.

First:

$$MAPE = \frac{1}{(L+1)N} \sum_{l=0}^{L} \sum_{j=1}^{N} \frac{|\tilde{c}_{t_l}(\tau_j, m_j) - G_{0j} - \sum_{i=1}^{d} G_{ij}\xi_{i,t_l}|}{\tilde{c}_{t_l}(\tau_j, m_j)}$$

This is the summation over time and calls of the normalized error between actual prices and reconstructed prices. The aim is to minimize the statistical error between actual Heston prices and the prices we get via the decomposition in latent factors.

$$PDA = 1 - \frac{tr(\Lambda^{-1} G \tilde{Z}^\top \tilde{Z} G^\top \Lambda^{-1})}{tr(\tilde{Z}^\top \tilde{Z})}$$

The numerator represents the variance of z_t explained by the factors. The denominator represents the total variance of z_t. As z_t is the drift term necessary to get absence of dynamic arbitrage, this metric measures how well we mninimize dynamic arbitrage, i.e. how close z_t is to be in the generated column space of G^T (which we recall is the ideal case, but impossible to achieve).

$$PSAS = 1 - \frac{\sum_{l=0}^{L} \mathbf{1}_{\{AG^\top \xi_{t_l} \geq b\}}}{L+1}$$

The right term is the proportion of timesteps where the no static arbitrage constraints are respected. We want a PSAS close to 0.

It turns out that with only one factor for static arbitrage, and one factor for dynamic arbitrage, we obtain the following results: 3.85% MAPE, 3.04% PDA, 0.37% PSAS. Hence it is enough to only use 3 factors in total (including the stock price). That means that even if the neural networks perfectly approximate the drift and diffusion of the price and latent factors, there will still be a structural error left (3.85 % on the data used to construct the Heston prices).

This is the first validation step: we make sure that we can get a good approximation of option prices with this method.
However, the goal is not to reconstruct prices from the past, but to simulate future prices. In this context, we won't have the Heston prices to decode the factors from, hence we will need to simulate trajectories of such factors: that is when the Neural SDE comes into action.

Dataset construction: The dataset consists of discrete-time observations of the underlying asset price S_t and the corresponding decoded factors by algorithm 1, already used to asses statistical precision. These are the inputs for both of the networks. The data are split chronologically, with 90% of the observations used for training and the remaining 10% reserved for validation.

7.4 Training Phase (Learning the Dynamics)

For a given t, each network takes as inputs $S_t, \xi_{1_t}, \xi_{2_t}$ and outputs $\mu(S_t, \xi_{1_t}, \xi_{2_t}), \sigma(S_t, \xi_{1_t}, \xi_{2_t})$. The goal is to learn the parameters θ that maximize the likelihood of the entire observed trajectory. The global loss function $J(\theta)$ is defined as the sum of the negative log-likelihoods over the entire training set:

$$J(\theta) = \sum_{t \in \text{Training Set}} -\log\left(p\left(\tilde{\xi}_{t+1} \mid \tilde{\xi}_t; \theta\right)\right)$$

Although the objective is the global sum, the optimization is performed iteratively using mini-batches:

- **Shuffling:** Transitions $(\tilde{\xi}_t, \tilde{\xi}_{t+1})$ are shuffled and not necessarily processed in chronological order.
- **Batch Update:** At each iteration, a small subset (batch) of transitions is selected. The loss is computed for each point in this batch, and we do the average for the points in the batch. The gradients are computed on this batch to update the weights using the Adam optimizer. Then we set this batch apart and we start again with another batch, until the model

has seen all of the training data. This is one epoch. We then start again with a different shuffle of the training set, until we reach the total number of epochs.

For a given transition, the loss is derived from the Euler-Maruyama approximation of the transition density, as explained in section 6.

Structural Constraints: The network architecture imposes linear inequality constraints on the outputs μ_θ and σ_θ. This ensures that regardless of the weight updates during backpropagation, the resulting dynamics always remain within the arbitrage-free polytope, as described in section 6.

8 Validation and tests

8.1 In-Sample Test

Once the training is complete, the parameters θ are frozen. The model is then run on the training dataset to verify if it successfully "reverse-engineered" the Heston dynamics. The estimated local dynamics (Neural SDE outputs) are compared against the theoretical "Ground Truth" derived from the Heston equations. Namely, for each timestamp in the training set, we compare:

- **For the Stock S_t:** we directly compare $\mu^{S,\theta}(S_t, \xi_{1_t}, \xi_{2_t})$ and $\sigma^{S,\theta}(S_t, \xi_{1_t}, \xi_{2_t})$ with the true Heston drift (0) and diffusion of the stock price S, which is at time t: $(\mathcal{L}(t,S)\sqrt{\nu_t}S_t)$.
- **For Factor ξ_1:** In that case, we don't have a true "ground truth" model to compare the drift and diffusion obtained with the neural SDE, since in the Heston model there are no dynamics for such hidden factor, but only for the volatility. However, by plotting the volatility of Heston and the first hidden factor, both evaluated on the training set, one can see a strong linear relationship. Then one can write $\nu = \beta_0 + \beta_1\xi_1$, and apply Itô's formula to the Heston variance using this linear relation to get:

$$\mathrm{d}\xi_{1t} = \mu^{\xi_1}(\xi_{1t})\,\mathrm{d}t + \sigma^{\xi_1}(\xi_{1t})\,\mathrm{d}W_t^{\nu}, \quad \mu^{\xi_1}(x) = \frac{\kappa(\theta - \beta_0 - \beta_1 x)}{\beta_1}, \quad \sigma^{\xi_1}(x) = \frac{\eta\sigma\sqrt{\beta_0 + \beta_1 x}}{\beta_1}.$$

Hence we can compare the drift and diffusion obtained thanks to the neural SDE model with the functions μ^{ξ_1} and σ^{ξ_1}

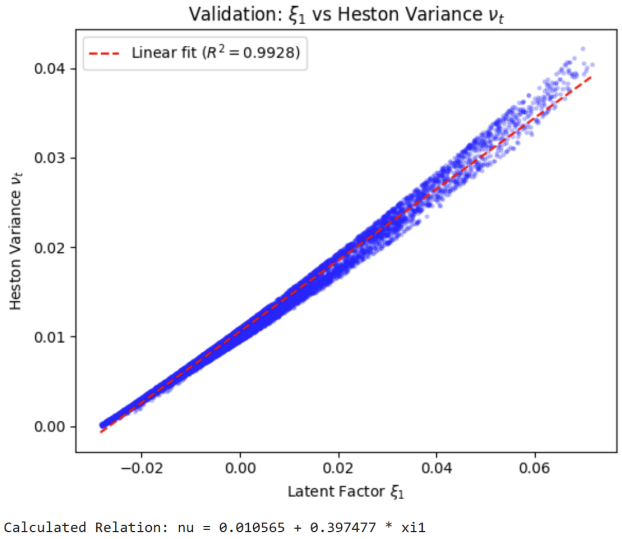


Figure 2: Linear relation between the volatility and the first latent factor

- **For Factor ξ_2:** No ground truth exists (as Heston has only 1 variance factor), hence we can't compare the second factor to anything. We will come back to this problem at the end of the section.

  This is the second validation step: we check that the neural SDE has accurately learnt the *local dynamics* (drift and diffusion functions) of the Heston model. This ensures that the model captures the correct transition laws, which is a prerequisite for generating realistic trajectories and consistent option prices.

In the original paper, the validation relies on **Scatter Plots**:

- **X-axis:** Theoretical Ground Truth value (from Heston equations).
- **Y-axis:** Neural Network estimate (evaluated at the same state).
- **Success Criterion:** Points should align along the diagonal $y = x$.

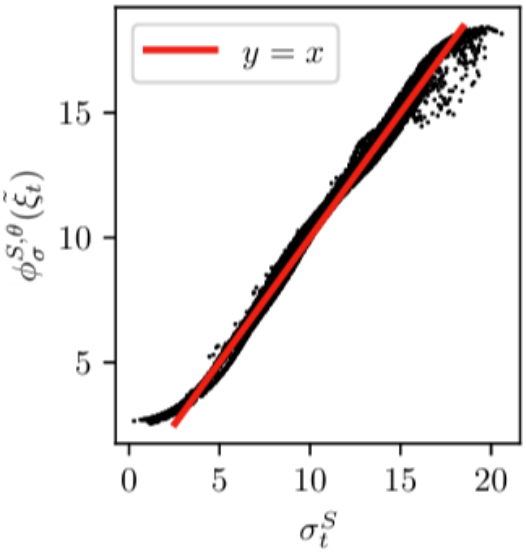


Figure 3: Heston vs Neural SDE for diffusion

Results show a near-perfect alignment for diffusion terms, confirming the model effectively learned the local volatility structure directly from the trajectory data.

8.2 Out-of-sample test

The out-of-sample test, distinct from the in-sample validation performed on the training set, assesses the Neural SDE model's capacity to function as an autonomous market generator. Unlike the in-sample test, we do not have a pathwise ground truth to compare directly against the simulated trajectories. Instead, we validate that the generated trajectories exhibit statistical properties and distributions consistent with the input data (Heston-SLV), as detailed below.

Unlike the training phase, the simulation does not rely on external input data to predict the next step. Instead, the generation is recursive:

1. At time t, the current simulated state $\tilde{\xi}_t = (S_t, \xi_{1,t}, \xi_{2,t})$ is fed into the neural networks.
2. The networks output the local drift $\mu(\tilde{\xi}_t)$ and diffusion $\sigma(\tilde{\xi}_t)$ coefficients specific to this state.
3. The next state $\tilde{\xi}_{t+\Delta t}$ is computed using a tamed Euler scheme.
4. This new state becomes the input for the subsequent step, creating a self-sustaining trajectory independent of the path generated in the training set.

Remark:
Although the *out-of-sample* simulation starts from a point potentially different from that of the historical data, the comparison of statistical properties remains relevant for the following reasons:

- **Simulation horizon and ergodicity:** Validation relies on a long simulation (10,000 time steps). Over this horizon, the empirical distributions are compared, indicating that the evaluation focuses on the stationary distribution of the process rather than its position at a time t dependent on the initial state.
- **Mean-reverting dynamics:** The reference model (Heston-SLV) imposes mean-reverting dynamics for the variance ($\kappa(\theta - \nu_u)$). Consequently, the influence of the starting point on the distribution of values fades over the long term, validating the approach of comparing global histograms.

Benchmarking against Ground Truth: Since the simulation generates entirely new pathwise realizations, point-to-point comparison with historical data is impossible. Validation therefore relies on comparing the statistical properties (marginal and joint distributions) of the simulated trajectories against the original input dataset.

- **For the asset price S_t:** The simulated distributions are compared to the trajectories generated by the Heston-SLV model.

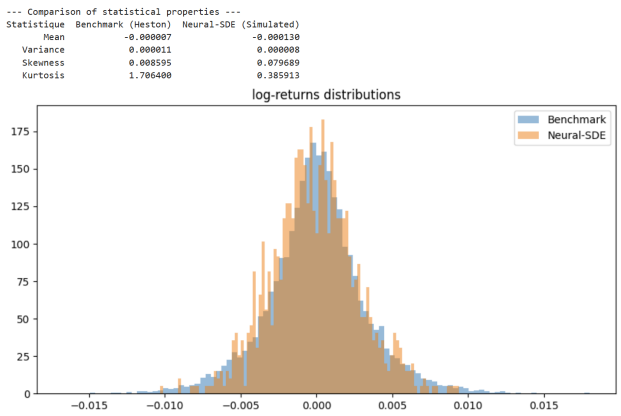


Figure 4: Statistical properties of Heston-SLV vs Neural SDE

- **For the latent factors ξ_t:** Since the standard Heston-SLV model does not explicitly model these factors, the "ground truth" reference is constructed by decoding the original Heston-SLV option prices using the factor construction algorithm (Algorithm 1) . The simulated factors are thus compared against these pre-processed, decoded factors from the input dataset.

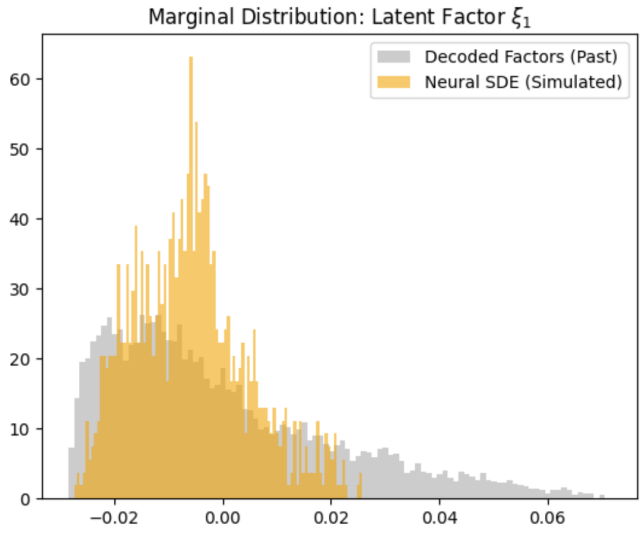


Figure 5: Distributions of the factor ξ_1

This is the third validation step: we check that the network can re-

produce the statistical properties and stylized facts of the ground truth Heston process, assessing that the model did not just memorize the local transitions on the training data but learned the global dynamics.

Pricing and Consistency Checks: The ultimate goal being option pricing, the simulated latent factors are mapped back to the price surface using the linear factor representation described earlier:

$$\tilde{c}_t(\tau, m) = G_0(\tau, m) + \sum_{i=1}^{d} G_i(\tau, m)\xi_{i,t}$$

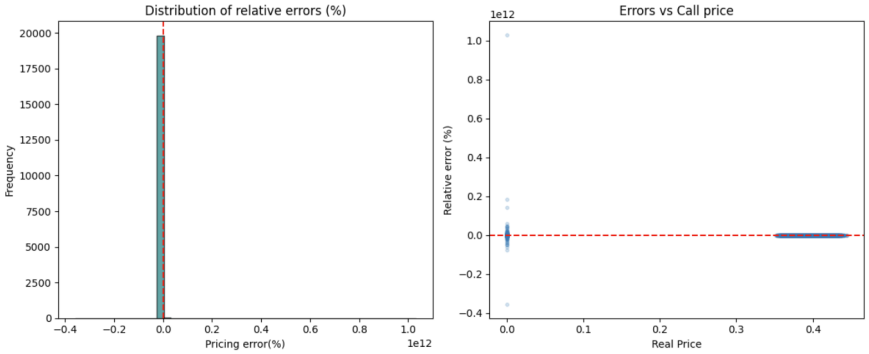


Figure 6: Analysis of the pricing performance

The analysis of the residual plots reveals several key insights into the model's calibration:

- **Main Convergence Zone:** A dense cluster of residuals is observed for option prices ranging between 0.3 and 0.4. This region typically corresponds to At-the-Money (ATM) options. In this area, the model exhibits high stability and homoscedasticity, with relative errors tightly centered around the zero-axis, confirming the network's ability to learn the core pricing function.
- **Numerical Instability near the Origin:** A vertical clustering of errors is visible as the real price approaches zero. This "perpendicular" artifact is a direct consequence of the relative error metric ($\frac{\hat{y}-y}{y}$). For Deep Out-of-the-Money (OTM) options with near-zero theoretical values, even a negligible absolute pricing discrepancy results in an explosive relative error.

Remark on generalisation: One could think that the Neural SDE is trained to be a pale copy of Heston SLV. However, this just comes from the fact that in

the original paper, the authors trained the model on Heston, considering it as a ground truth. If we train it on real data, the Neural SDE will then consider this data as ground truth and then reproduce "real trajectories". If we want to forecast real option prices, we should follow this procedure:

Initialization with Real Data (Practical Application) If you want to use this model tomorrow on the real market, Bloomberg does not provide "initial values" for the latent factors ξ. You must derive them yourself. The procedure is as follows:

1. You retrieve the market option prices at time t
2. You solve for the values $\xi_{1,t}, \xi_{2,t}$ which, when passed through the "Decoder" (the linear combination of basis functions G), reproduce these market prices.

$$\min_{\xi} \|\text{Market Price} - \text{Model Price}(\xi)\|^2$$

3. This optimization yields your starting point $(\xi_{1,t}, \xi_{2,t})$.
4. From this initial state, you launch the Neural SDE (previously trained on historical data) to simulate potential future scenarios.

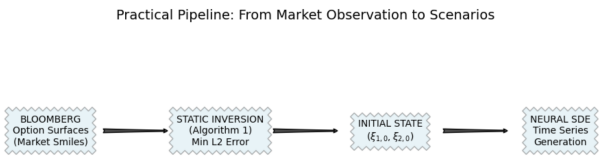


Figure 7: Pipeline for practical implementation

The training phase on real market data follows a similar logic to the initialization, but applied historically:

1. **Dataset Construction (Daily Calibration):** You collect historical option price surfaces over a long period (e.g., 10 years). For every single day t in your history, you apply the calibration step described above.
2. **Factor Time Series:** This converts your database of option prices into a time series of realized latent factors: $\{(\xi_{1,t}, \xi_{2,t})\}_{t=1}^{T}$.
3. **Training the Neural SDE:** You then feed this derived time series of factors into the neural network. The network learns the transition laws (drift and diffusion) that explain how the market moved from the factors at day t to the factors at day $t+1$ throughout history.

8.3 Critique of the approach

The inclusion of a second latent factor, ξ_2, in the Neural SDE model is necessitated by the structural limitations of the dimensionality reduction technique rather than the intrinsic physics of the Heston benchmark. Although the Heston model relies on a single source of variance (ν_t), the relationship between this variance and the implied volatility surface is highly non-linear. Since the authors employ a linear decoder (based on PCA) to map the latent state to option prices, a single linear factor is insufficient to minimize the reconstruction error of the curved price surface. Consequently, ξ_2 is required as an auxiliary coordinate to capture these non-linearities within a linear subspace.

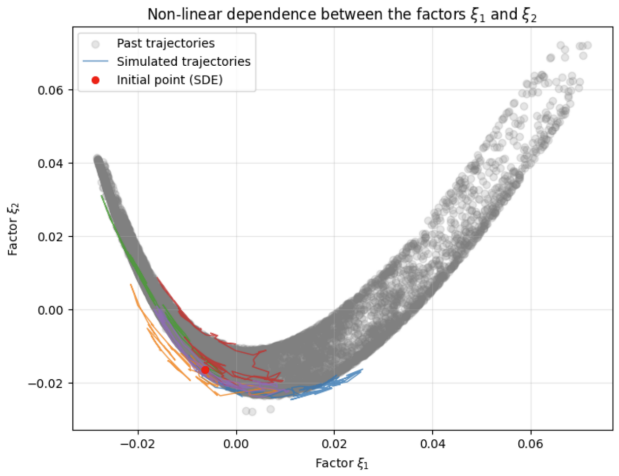


Figure 8: Polynomial relationship between the two factors

— Dependency Metrics — Pearson Correlation (Linear) : -0.0558 Spearman Correlation (Rank) : -0.4203 Polynomial R^2 (Degree 2) : 0.8827

However, the "In-sample" analysis reveals a strong polynomial dependence between the factors (R^2 of 0.8827) indicating that ξ_2 is dynamically redundant and carries no independent stochastic information. This artifact complicates the physical validation of the model, as there is no theoretical "ground truth" drift or diffusion for ξ_2 in the Heston equations. A potentially superior approach would be to replace the linear PCA step with a non-linear autoencoder. Such an architecture could theoretically compress the state into a single latent dimension ($d = 1$), thereby aligning the learned Neural SDE perfectly with the true one-factor structure of the underlying Heston dynamics.

References

[1] John C. Hull. *Options, Futures, and Other Derivatives*. 4th. Pearson, 2000. ISBN: 978-0136939979.

[2] Midhun T Augustine. *A Survey on Universal Approximation Theorems*. 2024. arXiv: 2407.12895 [cs.LG]. URL: https://arxiv.org/abs/2407.12895.

[3] Patryk Gierjatowicz et al. *Robust pricing and hedging via neural SDEs*. 2020. arXiv: 2007.04154 [q-fin.MF]. URL: https://arxiv.org/abs/2007.04154.

[4] Lei Fan and Justin Sirignano. *Machine Learning Methods for Pricing Financial Derivatives*. 2024. arXiv: 2406.00459 [q-fin.CP]. URL: https://arxiv.org/abs/2406.00459.

[5] Samuel N. Cohen, Christoph Reisinger, and Sheng Wang. "Detecting and Repairing Arbitrage in Traded Option Prices". In: *Applied Mathematical Finance* 27.5 (Sept. 2020), pp. 345–373. ISSN: 1466-4313. DOI: 10.1080/1350486x.2020.1846573. URL: http://dx.doi.org/10.1080/1350486X.2020.1846573.

[6] Marc Yor et al. "Stochastic volatility for Lévy processes". In: *Mathematical Finance* 13 n.3 (2003), pp. 345–382. URL: https://hal.science/hal-00103904.

[7] Douglas T. Breeden and Robert H. Litzenberger. "Prices of State-Contingent Claims Implicit in Option Prices". In: *The Journal of Business* 51.4 (1978), pp. 621–651. ISSN: 00219398, 15375374. URL: http://www.jstor.org/stable/2352653 (visited on 02/17/2026).

[8] Samuel N. Cohen, Christoph Reisinger, and Sheng Wang. *Arbitrage-free neural-SDE market models*. 2021. arXiv: 2105.11053 [q-fin.CP]. URL: https://arxiv.org/abs/2105.11053.

[9] Ian T. Jolliffe and Jorge Cadima. "Principal component analysis: a review and recent developments". In: *Philosophical Transactions of the Royal Society A: Mathematical, Physical and Engineering Sciences* 374.2065 (Apr. 2016), p. 20150202. ISSN: 1364-503X. DOI: 10.1098/rsta.2015.0202. eprint: https://royalsocietypublishing.org/rsta/article-pdf/doi/10.1098/rsta.2015.0202/1381479/rsta.2015.0202.pdf. URL: https://doi.org/10.1098/rsta.2015.0202.

[10] Avner Friedman and Mark A. Pinsky. "Asymptotic Stability and Spiraling Properties for Solutions of Stochastic Equations". In: *Transactions of the American Mathematical Society* 186 (1973), pp. 331–358. ISSN: 00029947. URL: http://www.jstor.org/stable/1996569 (visited on 02/17/2026).